2025

ONLINECASINO

MEISER MARCEL
OSKAR RICHERT
KILIAN SCHELLENBERG

BETREUER
HANS APFELBECK; MATTHIAS GRAF

EDV SCHULE PLATTLING | 94447 Plattling

Inhalt

1.Einleitung 2
1.1 Firmenprofil 2
1.2 Projektumfeld 2
1.3 Ist-Analyse 3
1.4 Soll-Zustand 3
1.5 Zeitplan 4
2. Entwurf / Planung 5
3. Umsetzung 6
3.1 Projektvorbereitung 6
3.2 Backend-Entwicklung (C#) 6
3.2.1 Einrichtung des Webservice 6
3.2.2 API-Endpunkte zur Kommunikation mit dem Frontend 7
3.2.3 Anbindung an Datenbank 8
3.2.4 Sicherheit 9
3.3 Frontend-Entwicklung (HTML, JavaScript) 11
3.3.1 Aufbau der Benutzeroberfläche 11
3.3.2 Dynamische Darstellung von Spielkomponenten 12
3.3.3 Anbindung an die API 13
3.4 Deployment und Veröffentlichung 14
4. Qualitätssicherung / Tests 15
5.Fazit / Ausblick 15
5.1 Fazit 15
5.2 Mögliche Erweiterungen 16
Abbildungsverzeichnis 16

1.Einleitung

1.1 Firmenprofil

Die EDV-Schulen des Landkreises Deggendorf in Plattling sind seit dem Jahr 1987 eine anerkannte Berufsfachschule für IT-Ausbildungen. Innerhalb von drei Jahren werden die Schülerinnen und Schüler praxisnah sowie projektorientiert zu IHK-zertifizierten Fachinformatikerinnen und Fachinformatikern ausgebildet. Dabei haben sie die Möglichkeit, sich entweder auf die Fachrichtung Anwendungsentwicklung oder Systemintegration zu spezialisieren.

Im zweiten Ausbildungsjahr (11. Klasse) bearbeiten die Lernenden in Gruppen von zwei bis drei Personen ein fachbezogenes Projekt, das im Anschluss schriftlich dokumentiert und im Rahmen einer Präsentation vorgestellt wird. Diese Projektarbeit fördert sowohl die Teamfähigkeit als auch die praxisnahe Anwendung der erlernten Inhalte und bereitet die Auszubildenden gezielt auf die Anforderungen im Berufsleben vor.

1.2 Projektumfeld

Das Projekt „Onlinecasino“ wurde im Rahmen der schulischen Ausbildung an der EDV-Schule Plattling umgesetzt. Ziel des Projekts ist die Entwicklung einer webbasierten Anwendung, die verschiedene Funktionen eines Onlinecasinos simuliert und dabei moderne Softwaretechnologien und Entwicklungsmethoden nutzt.

Programmiersprachen:

- C#
- JavaScript
- HTML, CSS
- T-SQL (für die Datenbank)

Frameworks/Technologien im Backend:

- .NET 8 (C#, ASP.NET Core Web API)
- Entity Framework Core (ORM für Datenbankzugriffe)
- Microsoft.AspNetCore.Authentication.JwtBearer (für Authentifizierung via JWT)
- Swashbuckle.AspNetCore (für OpenAPI/Swagger Dokumentation)

Frameworks/Technologien im Frontend:

- TailwindCSS – Utility-first CSS-Framework für Design.
- DaisyUI – Komponentenbibliothek auf Basis von TailwindCSS.
- SweetAlert – JavaScript-Framework für benutzerfreundliche Popups.

Weitere Technologien:

- SQL-Server als relationale Datenbank

Schnittstellen:

- REST-API (z.B. Controller wie UserController, Endpunkte für Login, Registrierung, Credits)
- Datenbankschnittstelle via Entity Framework an Microsoft SQL Server
- Zahlungsschnittstelle: PayPal Sandbox (für Testzahlungen)
- E-Mail-Versand: MailJS

Tools:

- Entwicklungsumgebung: VS-Code
- Versionsverwaltung: Git, GitLab

1.3 Ist-Analyse

Zu Projektbeginn lagen keine vorhandenen Softwarekomponenten oder technischen Grundlagen vor. Es standen weder Quellcode noch Datenbankschemata, Schnittstellen oder Dokumentationen zur Verfügung. Die gesamte Lösung wurde im Rahmen des Projekts neu konzipiert und realisiert.

Sämtliche Entscheidungen hinsichtlich Systemarchitektur, Technologieauswahl und Funktionsumfang konnten frei und bedarfsorientiert getroffen werden. Die Umsetzung umfasste alle typischen Phasen der Softwareentwicklung – beginnend mit der Anforderungsanalyse über die Planung und Konzeption bis hin zur Implementierung und abschließenden Dokumentation.

Durch die vollständige Neuentwicklung bestand keine Abhängigkeit zu bestehenden Systemen oder veralteten Technologien. Dadurch war der Einsatz aktueller Frameworks, Werkzeuge und etablierter Entwicklungsstandards möglich.

1.4 Soll-Zustand

Ziel des Projekts „Onlinecasino“ ist die Entwicklung einer webbasierten Casino-Anwendung, die die folgenden Anforderungen erfüllt:

- Bereitstellung von mindestens drei unterschiedlichen Casinospielen. Im Rahmen des Projekts wurden vier Spiele implementiert: Roulette, Blackjack, Risikoleiter und TripleGuess.
- Vollständige Erreichbarkeit der Anwendung innerhalb des Schulnetzwerks der EDV-Schule Plattling, einschließlich Frontend, Backend und Datenbankdienste.
- Sicherstellung eines stabilen und fehlerfreien Betriebs. Kritische Fehler oder Störungen, die den Spielablauf oder die Benutzererfahrung beeinträchtigen könnten, sind zu vermeiden.
- Integration einer Möglichkeit zur Guthabeneinzahlung über PayPal. Für diesen Zweck wird die PayPal Sandbox als sichere Testumgebung verwendet.

Die Umsetzung dieser Anforderungen gewährleistet eine funktionale, technisch zuverlässige und nutzbare Webanwendung im schulischen Umfeld.

1.5 Zeitplan

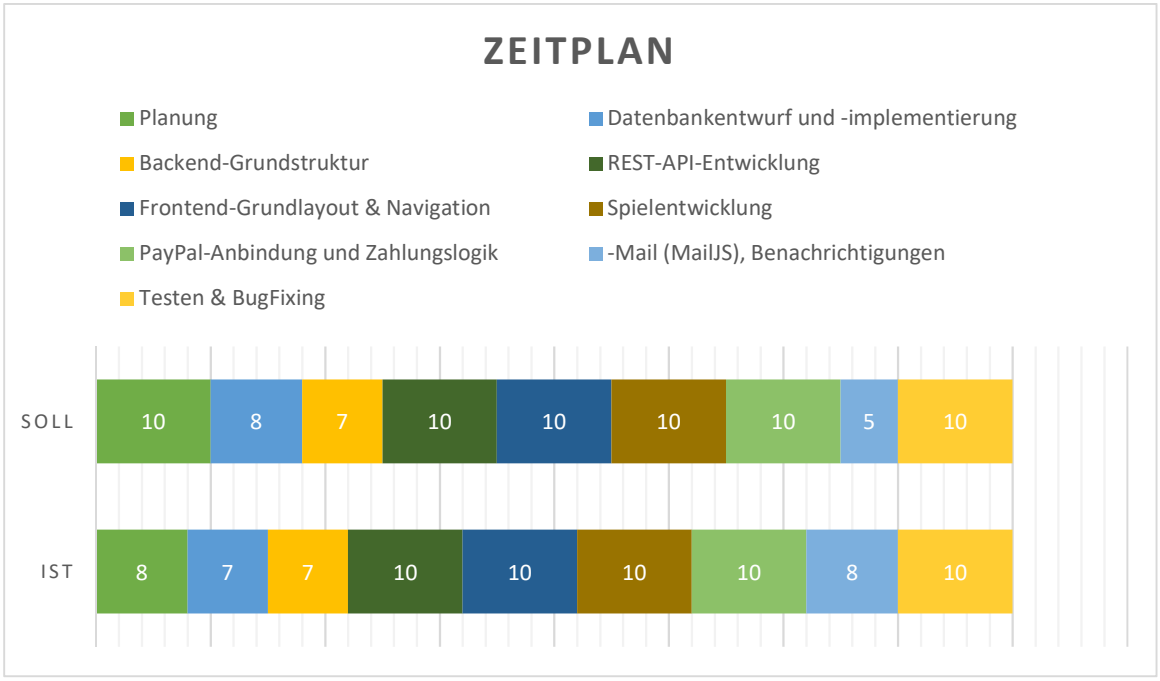


Abbildung 1: Zeitplan

- Die Gesamtarbeitszeit stimmt mit der Planung überein (80 Stunden).
- Weniger Zeit als geplant wurde für Planung und Datenbankentwurf benötigt (insgesamt 3 Stunden eingespart).
- Die E-Mail/Benachrichtigungs-Integration war aufwändiger als geplant (+3h).
- Alle anderen Arbeitspakete entsprechen exakt der Planung

2. Entwurf / Planung

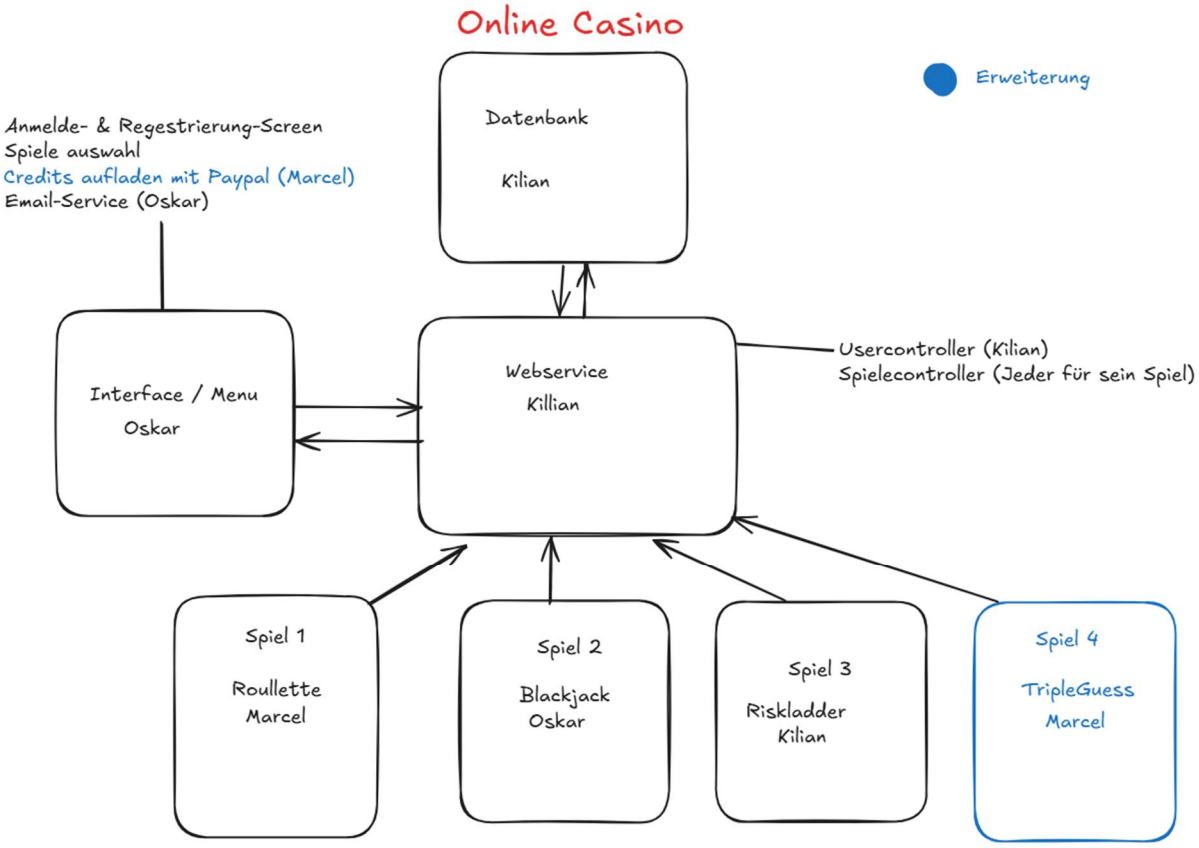


Abbildung 2 - Mockup

Die Abbildung 2 zeigt den modularen Aufbau des Online-Casino-Systems. Zentrale Komponente ist der Webservice, der die Kommunikation zwischen Benutzeroberfläche, Spielen und Datenbank steuert. Die Benutzeroberfläche (Oskar) ermöglicht Registrierung, Spielauswahl und enthält einen E-Mail-Service sowie eine Erweiterung zur PayPal-Creditaufladung (Marcel).

Die Spielemodule sind über eigene Spielecontroller an den Webservice angebunden.
Aktuell sind vier Spiele integriert: Roulette (Marcel), Blackjack (Oskar), Riskladder (Kilian) und als Erweiterung TripleGuess (Marcel). Die Benutzer- und Spieldaten werden über den Usercontroller verwaltet und in der Datenbank (Kilian) gespeichert.
Die klare Aufgabentrennung unterstützt eine effiziente Entwicklung und spätere Erweiterbarkeit des Systems.

3. Umsetzung

3.1 Projektvorbereitung

Für die Umsetzung des Projekts wurde ein zeitgemäßer Technologie-Stack gewählt. Die serverseitige Logik wurde mit C# unter Verwendung von .NET 8.0 und dem Framework ASP.NET Core Web API realisiert. Die Benutzer-Authentifizierung basiert auf dem JSON Web Token (JWT)-Verfahren, dessen Konfiguration zentral in der Datei **appsettings.json** gepflegt wird. Zur sicheren Speicherung von Passwörtern wurde die Bibliothek BCrypt.Net-Next eingesetzt. Der Zugriff auf die relationale Datenbank erfolgte über Entity Framework Core in Verbindung mit Microsoft SQL Server.

Zur Dokumentation und Testbarkeit der Schnittstellen kam Swagger über das NuGet-Paket Swashbuckle.AspNetCore zum Einsatz. Die Protokollierung wurde über die Logging-Funktionalität von ASP.NET Core angepasst und ermöglicht eine gezielte Analyse während der Entwicklungs- und Testphase.

Das Frontend wurde mit HTML und JavaScript umgesetzt und nutzt das moderne Tailwind CSS-Framework zur Gestaltung eines responsiven Layouts. Ergänzend wurde DaisyUI zur effizienten Einbindung vorgefertigter UI-Komponenten verwendet. Die Verwaltung externer Abhängigkeiten erfolgt über npm, wobei die Projektkonfiguration in der Datei **package-lock.json** dokumentiert ist.

Die Entwicklung der Backend- und Frontend-Komponenten erfolgte vollständig in Visual Studio Code. Für lokale Tests und den Betrieb der Anwendung kamen die integrierten Webserver Kestrel sowie IIS Express zum Einsatz.
Die Versionsverwaltung des Quellcodes wurde mit Git durchgeführt und über ein privates Repository auf der GitLab-Instanz der EDV-Schule Plattling unter https://gl.edvschule-plattling.de verwaltet.

3.2 Backend-Entwicklung (C#)

3.2.1 Einrichtung des Webservice

Der Webservice des Onlinecasinos wurde mit **ASP.NET Core Web API** in **C#** entwickelt. Er ist für die gesamte Logik auf der Serverseite zuständig. Die Einrichtung des Webservices erfolgt in der Datei **Program.cs**. Dort werden wichtige Funktionen wie Controller, Swagger (für die API-Dokumentation), **CORS** (für den Zugriff vom Frontend) und die **JWT-Authentifizierung** aktiviert und konfiguriert.

Zur Laufzeit startet die Anwendung über app.Run() und stellt sämtliche API-Endpunkte unter einer gemeinsamen URL-Struktur (z. B. /api/user/...) zur Verfügung. Die Routing-Definition erfolgt dabei über das Attribut [Route("api/...")] direkt in den jeweiligen Controllern.

Zusätzlich wurde die API mit **Swagger (OpenAPI)** dokumentiert. Über den Swagger-UI-Endpunkt können alle verfügbaren Schnittstellen inklusive der HTTP-Methoden und erforderlichen Parameter bequem getestet und nachvollzogen werden. Auch die Übergabe eines JWT-Tokens ist hier möglich, um geschützte Endpunkte zu testen.

Die Swagger-Konfiguration mit integrierter JWT-Unterstützung ist in Abbildung 3 dargestellt:

```
// JWT-Authentifizierung zu Swagger hinzufügen
c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
{
    Name = "Authorization", // Name des Headers
    Type = SecuritySchemeType.ApiKey, // Typ: API Key (da Header)
    Scheme = "Bearer", // Schema: Bearer
    BearerFormat = "JWT", // Format: JWT
    In = ParameterLocation.Header, // Wo der Token eingefügt wird
    Description = "Gib dein JWT Token hier ein: 'Bearer {token}'"
});
```

Abbildung 3 - Swagger Authorization

Die Kommunikation zwischen Backend und Frontend erfolgt standardisiert über **RESTful HTTP-Endpunkte** mit JSON als Datenformat.

3.2.2 API-Endpunkte zur Kommunikation mit dem Frontend

Damit jedes Spiel eigenständig funktioniert, besitzt es einen eigenen Controller. Dieser ist dafür zuständig, die benötigten Spieldaten bereitzustellen und an das Frontend zu übermitteln.
Das folgende Beispiel zeigt einen Ausschnitt aus dem **TripleGuessController**. Hier wird demonstriert, wie eine zufällige Spielkarte generiert und über eine HTTP-Schnittstelle an den Client gesendet wird.

```
37      [HttpGet("newCard")]
        1 reference
38      public IActionResult newCard()
39      {
40          var rnd = new Random();
41          string suit = Suits[rnd.Next(Suits.Length)];
42          var valueTuple = Values[rnd.Next(Values.Length)];
43          string imageUrl = $"/Picture/Karten/{valueTuple.Name}_of_{suit}.png"; //Pfad für Karten generieren
44
45          var card = new
46          {
47              Name = valueTuple.Name,
48              Wert = valueTuple.Value,
49              Art = suit,
50              Bild = imageUrl
51          };
52          if (cardCache.Contains(card))
53          {
54              newCard();
55          }
56          cardCache.Add(card);
57          return Ok(card);
58      }
```

Abbildung 4- TripleGuess newCard Aufruf

Die in Abbildung 4 dargestellte Methode **newCard()** ist Bestandteil eines Webservice-Endpunkts, der über das Attribut **[HttpGet("newCard")]** gekennzeichnet ist. Dadurch kann der Client per HTTP GET eine neue Spielkarte anfordern. Die Methode stellt sicher, dass eine noch nicht gezogene Karte zurückgegeben wird.

1. Aufruf über HTTP GET

Die Methode ist über die Route **/newCard** aufrufbar. Sie gibt ein JSON-Objekt zurück, das die Eigenschaften der gezogenen Karte enthält. Dies ist für die clientseitige Anzeige und Spiellogik erforderlich.

2. Zufällige Kartenauswahl

Ein Objekt der Klasse Random generiert eine zufällige Kartenfarbe (suit) sowie einen zufälligen Kartenwert (valueTuple). Aus diesen beiden Werten wird anschließend die URL zum Kartenbild zusammengesetzt. Das Bild befindet sich im Verzeichnis /Picture/Karten/ und folgt dem Namensschema **{Wert}_of_{Farbe}.png**. Diese URL wird im JSON-Objekt an den Client übermittelt.

3. Vermeidung von Duplikaten mittels Karten-Cache

Um sicherzustellen, dass keine Karte doppelt gezogen wird, wird ein Zwischenspeicher (**cardCache**) verwendet. Vor dem Hinzufügen der Karte in diesen Cache erfolgt eine Prüfung, ob die Karte bereits enthalten ist. Sollte dies der Fall sein, wird die Methode erneut aufgerufen, bis eine eindeutige Karte gefunden wird.

4. Hinzufügen und Rückgabe der Karte

Sobald eine gültige, noch nicht gezogene Karte gefunden wurde, wird sie dem Cache hinzugefügt und anschließend mit **return Ok(card)** an den Client zurückgegeben.

3.2.3 Anbindung an Datenbank

Die Datenanbindung des Onlinecasinos erfolgt über **Entity Framework Core** in Kombination mit einem **Microsoft SQL Server**. Der sogenannte **DbContext** (OnlineCasinoContext) definiert die Verbindung zur Datenbank sowie die verfügbaren Tabellen als sogenannte DbSets – in unserem Fall insbesondere die Tabelle Users.

Die Konfiguration erfolgt im Code über die Methode OnConfiguring(), wobei eine Verbindungszeichenfolge verwendet wird, die Serveradresse, Datenbankname und Anmeldedaten enthält (siehe Abbildung 5):

Webservice > Database > OnlineCasinoContext.cs > OnlineCasinoContext

```
using System;
using Microsoft.EntityFrameworkCore;

namespace onlinecasino.Database
{
    // Der Datenbank-Kontext für die Anwendung – verwaltet die Verbindung zur Datenbank und die Tabellen (DbSets)
    7 references
    public class OnlineCasinoContext : DbContext
    {
        // Diese Eigenschaft stellt die Tabelle 'Users' in der Datenbank dar
        8 references
        public DbSet<User> Users { get; set; }

        // Hier wird konfiguriert, wie sich die Anwendung mit der Datenbank verbindet
        0 references
        protected override void OnConfiguring(DbContextOptionsBuilder options)
        {
            // Verbindungszeichenfolge für den SQL Server
            options.UseSqlServer("Server=10.0.104.73;Database=bfs2023ik_kschelle;User Id=kschelle;Password=abc;Encrypt=True;TrustServerCertificate=True;");
        }

        // Konfiguration des Datenmodells (z. B. Tabellennamen, Schema etc.)
        0 references
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Die Entität 'User' wird der Tabelle "Users" im Schema "Casino" zugeordnet
            modelBuilder.Entity<User>().ToTable("Users", schema: "Casino");
        }
    }
}
```

Abbildung 5- Datenbankzugriff

Im Code erfolgt der Datenbankzugriff objektorientiert, z. B. um Benutzer anhand des JWT-Tokens zu identifizieren:
var username = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
var user = await context.Users.FirstOrDefaultAsync(u => u.Username == username);
Auch Änderungen – etwa an den Credits nach einem Spiel – erfolgen direkt über das Objekt:
user.Credits += 50;
await context.SaveChangesAsync();
Somit werden **SQL-Operationen vollständig in C# gekapselt**, was die Lesbarkeit und Wartbarkeit verbessert und typische Sicherheitsprobleme wie SQL-Injection vermeidet.

3.2.4 Sicherheit

Das Sicherheitssystem des Onlinecasinos basiert auf **JWT-Authentifizierung (JSON Web Tokens)**. Bei erfolgreicher Anmeldung eines Nutzers wird ein verschlüsseltes Token generiert, das im Authorization-Header aller weiteren HTTP-Anfragen mitgesendet werden muss.

```
// Gibt Nutzerdaten zurück, sofern ein gültiger Token vorhanden ist
[Authorize]
[HttpGet("getData")]
0 references
public async Task<IActionResult> GetData()
{
```

Abbildung 6 - Authorization Header

Beim erfolgreichen Login wird ein JWT-Token erzeugt (siehe Abbildung 7):

```
var jwtKey = _configuration["Jwt:Key"];
var jwtIssuer = _configuration["Jwt:Issuer"];
var jwtAudience = _configuration["Jwt:Audience"];
var jwtExpire = int.Parse(_configuration["Jwt:ExpireMinutes"]!);

var claims = new[]
{
    new Claim(JwtRegisteredClaimNames.Sub, user.Username!),
    new Claim(JwtRegisteredClaimNames.Email, user.Email!),
    new Claim("UserId", user.Id.ToString()) // Benutzer-ID im Token speichern
};

var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey!));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var token = new JwtSecurityToken(
    issuer: jwtIssuer,
    audience: jwtAudience,
    claims: claims,
    expires: DateTime.Now.AddMinutes(jwtExpire),
    signingCredentials: creds
);
```

Abbildung 7 - JWT erzeugt

Das resultierende Token wird im Frontend gespeichert und bei zukünftigen Anfragen genutzt(siehe Abbildung 8):

```
const response = await axios.get(`http://${window.location.hostname}:5105/api/user/getData`, {
    headers: { Authorization: `Bearer ${token}` }
});
```

Abbildung 8 - Token Aufruf

Alle Benutzerpasswörter werden **nicht im Klartext gespeichert**, sondern mit dem enthaltenen Algorithmus **BCrypt** gehasht:
string hashedPassword = BCrypt.Net.BCrypt.HashPassword(data.Password);
Beim Login wird das eingegebene Passwort mit dem gespeicherten Hash verglichen:
BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash)
BCrypt bietet durch Salting und mehrere Iterationen einen hohen Schutz gegen Brute-Force- und Rainbow-Table-Angriffe.

Im Fall eines Passwortwechsels prüft das System zunächst das alte Passwort und ersetzt es dann durch den neuen Hash (siehe Abbildung 9):

```
if (!BCrypt.Net.BCrypt.Verify(request.OldPassword, user.PasswordHash))
{
    return BadRequest(new { message = "Das alte Passwort ist nicht korrekt." });
}

// Neues Passwort hashen und speichern
user.PasswordHash = BCrypt.Net.BCrypt.HashPassword(request.NewPassword);
await context.SaveChangesAsync();

return Ok(new { message = "Passwort erfolgreich geändert." });
```

Abbildung 9 - Verschlüsselung

Falls ein Benutzer sein Passwort vergessen hat, kann über den Endpunkt **forgotPassword** ein neues, zufälliges Passwort generiert werden (siehe Abbildung 10):

```
private string GenerateRandomPassword(int length)
{
    const string validChars = "ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!?";
    var random = new Random();
    return new string(Enumerable.Repeat(validChars, length)
        .Select(s => s[random.Next(s.Length)]).ToArray());
}
```

Abbildung 10 - Passwort erzeugen

Dieses wird dann gehasht in der Datenbank gespeichert und per E-Mail an den User geschickt, damit sich dieser wieder einloggen kann.

3.3 Frontend-Entwicklung (HTML, JavaScript)

3.3.1 Aufbau der Benutzeroberfläche

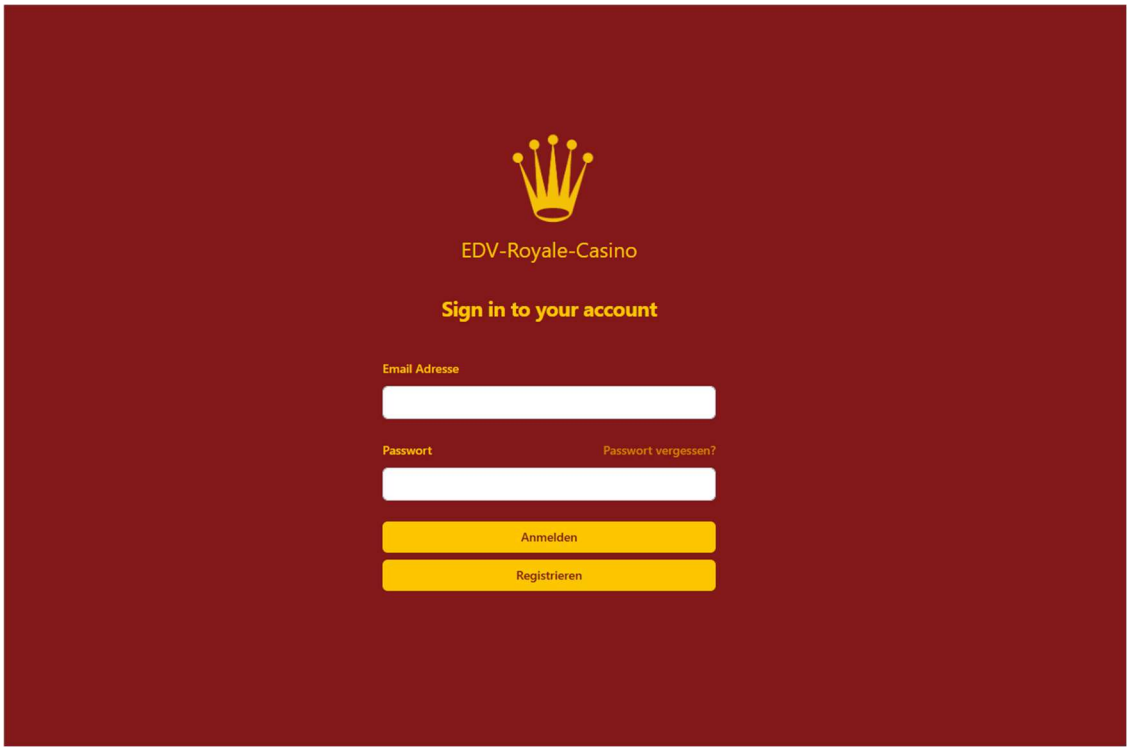


Abbildung 11 - Login-Formular des Online-Casinos

Die Benutzeroberfläche (siehe Abbildung 11) wurde mit TailwindCSS und DaisyUI umgesetzt. Sie bietet ein klares, casino-typisches Farbschema und benutzerfreundliche Elemente wie Eingabefelder und Buttons. Für Benachrichtigungen kommt SweetAlert zum Einsatz, um z. B. Fehler oder Login-Erfolge stilvoll anzuzeigen.

```
90    // Einlogen Methode
91    async function handleLogin(event) {  1
92      event.preventDefault();
93
94      const email = document.getElementById('email').value;
95      const password = document.getElementById('password').value;
96
97      try {
98    2   const response = await fetch(`http://${window.location.hostname}:5105/api/user/login`, {
99          method: 'POST',
100         headers: {
101           'Content-Type': 'application/json',
102         },
103         body: JSON.stringify({ email, password }),  3
104       });
105
106       if (response.ok) {
107         const data = await response.json();
108   4     const user = data.token;
109         localStorage.setItem("token", user);
110         window.location.href = 'index.html';
111       } else {
112         const errorText = await response.text();
113   5     alert('Fehler beim Login: ' + errorText);
114       }
115     } catch (error) {
116       alert('Verbindungsfehler: ' + error.message);
117     }
```

Abbildung 12 - Ablauf der Login-Funktion

Die Funktion handleLogin(event) (siehe Abbildung 12) dient der Benutzeranmeldung im Frontend und verarbeitet Eingabedaten clientseitig per JavaScript. Der Ablauf gestaltet sich wie folgt:

1. Zu Beginn wird das Standardverhalten des Formulars mit preventDefault() unterdrückt, um ein Neuladen der Seite zu verhindern.
2. Anschließend wird über fetch eine POST-Anfrage an die API http://<hostname>:5105/api/user/login gesendet. Die URL wird dynamisch anhand des aktuellen Hostnamens generiert.
3. Die E-Mail- und Passwortwerte aus den Eingabefeldern werden im JSON-Format im Body der Anfrage übermittelt.
4. Bei erfolgreicher Antwort (response.ok) wird das empfangene Token aus dem JSON-Objekt entnommen und im localStorage gespeichert. Danach erfolgt die Weiterleitung zur Startseite.
5. Im Fehlerfall wird eine Rückmeldung als Text angezeigt. Netzwerk- oder Serverfehler werden über einen catch-Block abgefangen und als Alert ausgegeben.

Diese Methode bildet die zentrale Schnittstelle zwischen Benutzeroberfläche und serverseitiger Authentifizierung und gewährleistet einen kontrollierten und abgesicherten Login-Prozess.

3.3.2 Dynamische Darstellung von Spielkomponenten

Ein besonderes Augenmerk wurde auf die visuelle und interaktive Darstellung des Roulette-Spiels gelegt. Der Roulette-Tisch wurde so umgesetzt, dass sich dieser dynamisch in eine Richtung dreht, während sich die Kugel gleichzeitig in entgegengesetzter Richtung bewegt – entsprechend der physikalischen Realität eines echten Roulette-Spiels. Dies sorgt für ein realistisches Spielerlebnis und unterstreicht die optische Qualität der Anwendung. Im Ausschnitt von Abbildung 13 ist diese Bewegung des Roulette Tisches sowie der Kugel dargestellt.

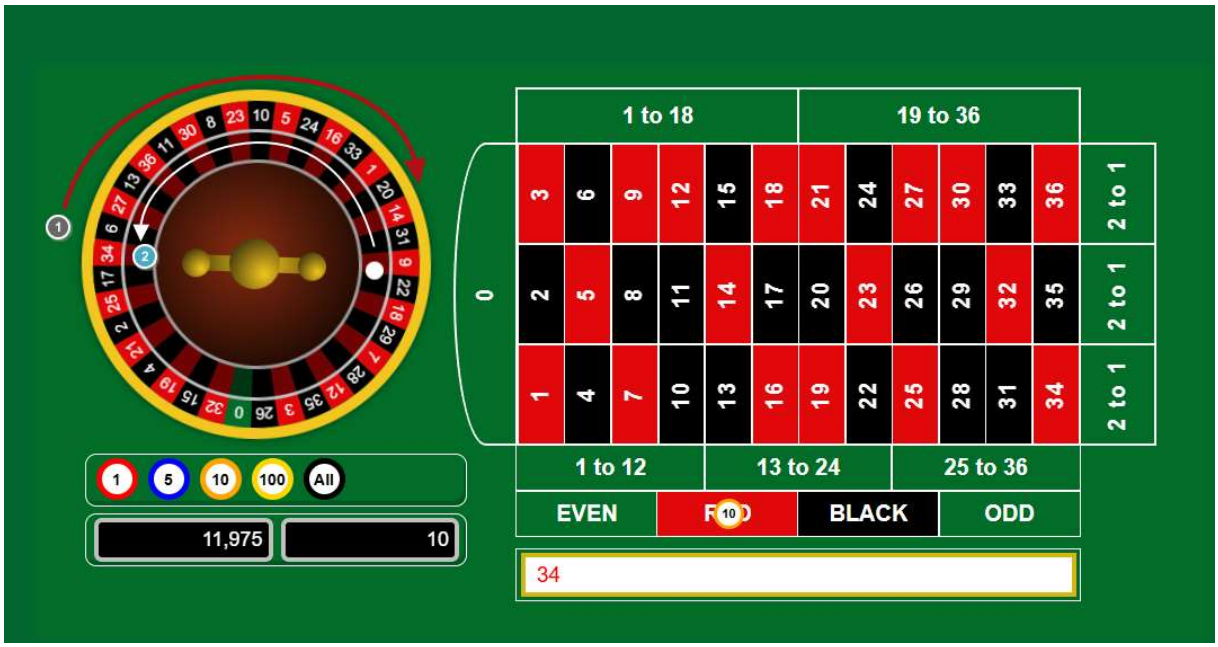


Abbildung 13 - Animation des Spiels Roulette

- **Pfeil 1** kennzeichnet die **Drehrichtung des Roulette-Tisches**,
- **Pfeil 2** zeigt die **Bewegung der Kugel**, welche sich entgegengesetzt zur Tischdrehung bewegt.

Die technische Umsetzung erfolgte über JavaScript unter Nutzung von CSS-Animationen, wobei die Bewegungseigenschaften gezielt gesteuert wurden, um ein flüssiges und visuell ansprechendes Ergebnis zu erzielen.

3.3.3 Anbindung an die API

Die Kommunikation zwischen dem Frontend und dem Backend erfolgt über HTTP-Anfragen, die mit der JavaScript-Bibliothek Axios realisiert werden. Axios ermöglicht es, asynchrone Webservice-Aufrufe an die im Backend bereitgestellten API-Endpunkte durchzuführen und die Daten im JSON-Format zu übertragen.

Im folgenden Beispiel wird gezeigt, wie der aktuelle Guthabenstand eines angemeldeten Benutzers vom Webservice abgefragt wird (siehe Abbildung 14):

```
14  async function fetchBankValue() { // holt sich Bankvalue
15      try {
16          // if (!token) await authorize();
17      1   const userData = await axios.get(`http://${window.location.hostname}:5105/api/user/getData`, {
18              headers: {
19                  Authorization: `Bearer ${token}`  2
20              }
21          });
22          return Number(userData.data.credits);
23      } catch (error) {
24          console.log("Fehler beim Laden des Bankwerts: ", error);
25          return 0;
26      }
27  }
```

Abbildung 14 - API-Aufruf mit Axios

- **Markierung 1**: Diese Variable enthält die Antwort des Servers auf den API-Aufruf. Die Antwort ist ein JSON-Objekt, das die aktuellen Nutzerdaten inklusive des Guthabens (credits) enthält. Das Guthaben wird aus dem Datenobjekt extrahiert und als Zahl zurückgegeben.
- **Markierung 2:** Zur Sicherstellung, dass nur authentifizierte Benutzer Zugriff auf die geschützten Endpunkte haben, wird ein sogenanntes JSON Web Token (JWT) im Authorization-Header der HTTP-Anfrage mitgeschickt. Dieses Token dient als Nachweis der erfolgreichen Anmeldung und wird vom Backend validiert.

Durch diese Methode wird sichergestellt, dass sensible Daten nur nach erfolgreicher Authentifizierung übertragen werden und die API-Aufrufe geschützt sind. Die Nutzung von Axios erlaubt zudem eine einfache Handhabung von asynchronen Webservice-Anfragen und Fehlerbehandlung.

3.4 Deployment und Veröffentlichung

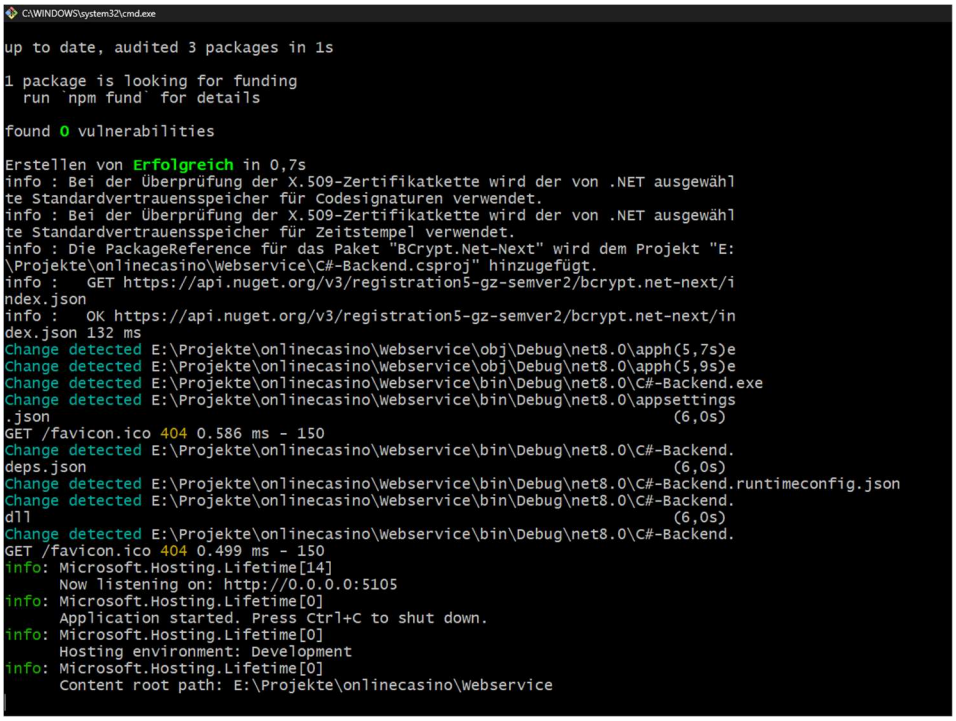


Abbildung 15 - Erfolgreicher Start des Webservices

Zum Starten des Webservices wurde ein Ausführungsskript direkt über die Kommandozeile ausgeführt (siehe Abbildung 15). Dieses Skript kompiliert das Backend-Projekt, lädt alle benötigten Abhängigkeiten (z. B. BCrypt.Net-Next) und startet anschließend den Webserver mit der definierten Konfiguration.

Im Screenshot ist zu sehen, wie das Skript die Anwendung erfolgreich baut und über den integrierten Kestrel-Webserver unter der Adresse http://0.0.0.0:5105 bereitstellt. Diese Adresse dient während der Entwicklung zum lokalen Testen der REST-API und zur Kommunikation mit dem Frontend.

Änderungen am Quellcode werden automatisch erkannt und führen zu einem sofortigen Neustart des Dienstes (Hot Reload), wodurch eine effiziente Entwicklungsumgebung entsteht.

Für das Frontend wurde zusätzlich der Live-Server von Visual Studio Code verwendet, um HTML-/JavaScript-Dateien ohne manuelles Neu laden direkt im Browser testen zu können.

4. Qualitätssicherung / Tests

Zur Sicherstellung der Softwarequalität wurden in allen Entwicklungsphasen manuelle Tests durchgeführt. Dabei wurden typische Fehlerszenarien, Randfälle sowie sicherheitsrelevante Aspekte gezielt geprüft. Alle erkannten Schwachstellen wurden dokumentiert und durch gezielte Codeanpassungen behoben.
Ein Beispiel aus der frühen Testphase betrifft das Roulette-Spiel: Es war möglich, während der Animation des Kugelwurfs weiterhin Einsätze zu platzieren. Dies konnte zu einer fehlerhaften Belastung des Spielerkontos und einem unfairen Vorteil führen. Zur Behebung dieses Problems wurde die Benutzerinteraktion mit dem Spielfeld, während der Drehanimation technisch blockiert und erst nach Abschluss der Animation wieder freigegeben.
Zusätzlich wurde das System von Dritten mit IT-Vorkenntnissen getestet. Die Rückmeldungen dieser externen Testpersonen führten zur Identifikation weiterer Fehlerquellen, insbesondere im Bereich der Spiellogik und Benutzerführung.
Die finale Anwendungsversion wurde nach allen Korrekturen erneut vollständig durchgetestet, um eine funktional stabile und benutzerfreundliche Ausführung zu gewährleisten.

5.Fazit / Ausblick

5.1 Fazit

Fazit von Marcel:

Im Rahmen des Projekts konnte ich meine bereits vorhandenen Kenntnisse im Umgang mit Git sowie in der Zusammenarbeit bei der Softwareentwicklung im Team weiter festigen. Besonders die strukturierte Arbeit mit Branches, Commits und Pull Requests im schulischen GitLab hat mir geholfen, versionssicher und effizient mit anderen zu arbeiten.

Darüber hinaus konnte ich mein Wissen in den Sprachen C#, JavaScript und HTML deutlich vertiefen. Die intensive Auseinandersetzung mit Frontend- und Backendtechnologien im Zusammenspiel hat mein technisches Verständnis nachhaltig gestärkt.

Ein besonders lehrreicher Aspekt war die Integration von PayPal als externe Zahlungsdienstleistung, da ich mich hierfür erstmalig mit API-Dokumentationen und der Anbindung von diesem Drittanbieterdiensten auseinandergesetzt habe.

Insgesamt hat mir das Projekt nicht nur fachlich neue Einblicke vermittelt, sondern auch meine Fähigkeit zur Zusammenarbeit und Problemlösung im Team weiterentwickelt.

Fazit von Oskar:

Während des Projekts konnte ich meine Kenntnisse im Frontend-Bereich deutlich ausbauen, insbesondere in JavaScript und HTML. Die Arbeit mit verschiedenen Frameworks und modernen Webtechnologien hat mein Verständnis für die Gestaltung und Umsetzung benutzerfreundlicher Oberflächen stark vertieft.

Zusätzlich habe ich grundlegende Erfahrungen mit C# sammeln können, vor allem bei der Implementierung von GET-Methoden für mein eigenes Spiel und der Kommunikation zwischen Frontend und Backend.

Die Zusammenarbeit im Team, insbesondere der koordinierte Umgang mit GitLab, hat mir gezeigt, wie wichtig klare Kommunikation und Versionskontrolle für den Projekterfolg sind.

Das Projekt hat mir nicht nur geholfen, mein fachliches Verständnis im Frontend-Bereich zu erweitern, sondern auch meine Kompetenzen im gemeinsamen Lösen von Problemen im Team deutlich verbessert.

Fazit von Kilian:

Im Rahmen des Projekts konnte ich vor allem meine Kenntnisse in der Backend-Entwicklung mit C# und ASP.NET Core vertiefen. Besonders die Arbeit mit Entity Framework Core zur Anbindung der SQL-Datenbank war sehr lehrreich, da ich mich intensiv mit Datenmodellen, Abfragen und der Integration von SQL-Strukturen in eine objektorientierte Umgebung auseinandergesetzt habe.

Ein zentrales Thema war zudem die Umsetzung eines sicheren Authentifizierungs- und Autorisierungskonzepts mittels JWT (JSON Web Tokens). Durch die Konfiguration der Token-Validierung, das Auslesen von Claims und die Absicherung geschützter Endpunkte habe ich ein solides Verständnis für moderne Sicherheitsmechanismen in Webanwendungen gewonnen.

Auch die Passwortsicherheit spielte eine wichtige Rolle. Die Verwendung von BCrypt zur Passwort-Hashung und die Implementierung einer Funktion zur Passwortzurücksetzung zeigten mir praxisnah, wie essenziell Datenschutz und Sicherheit bei der Entwicklung von Online-Plattformen sind.

Rückblickend konnte ich mein technisches Wissen in den Bereichen Datenbank, Sicherheit und Backend-Architektur deutlich erweitern und gleichzeitig meine Fähigkeit stärken, komplexe Anforderungen selbstständig und im Team umzusetzen.

5.2 Mögliche Erweiterungen

Zur Erweiterung des Systems könnte ein optionaler Onlineshop integriert werden, über den Nutzer kosmetische Inhalte wie alternative Profilbilder oder individuelle Kartendesigns erwerben können. Diese Ergänzungen bieten einen zusätzlichen Anreiz zur Nutzung und Personalisierung der Anwendung, ohne das eigentliche Spielgeschehen zu beeinflussen.

Eine weitere Verbesserung stellt der Betrieb der Anwendung in einem LXC-Container dar. Dies ermöglicht einen stabilen, durchgehend verfügbaren Betrieb und vereinfachten die Wartung sowie Skalierung des Systems. Um die Zugänglichkeit zu erhöhen, kann zusätzlich ein DNS-Eintrag (z. B. lxc6.dvs-plattling.de/appdns/) eingerichtet werden, sodass der Container über eine benutzerfreundliche URL statt einer IP-Adresse erreichbar ist.

Darüber hinaus lässt sich das System flexibel um neue Spiele erweitern. Durch die containerisierte Architektur bestehen hierbei keine technischen Beschränkungen hinsichtlich der Anzahl oder Art der hinzufügbaren Spiele.

Abbildungsverzeichnis

Abbildung 1: Zeitplan 4
Abbildung 2 - Mockup 5
Abbildung 3 - Swagger Authorization 7
Abbildung 4- TripleGuess newCard Aufruf 7
Abbildung 5- Datenbankzugriff 8
Abbildung 6 - Authorization Header 9
Abbildung 7 - JWT erzeugt 9
Abbildung 8 - Token Aufruf 10
Abbildung 9 - Verschlüsselung 10
Abbildung 10 - Passwort erzeugen 10
Abbildung 11 - Login-Formular des Online-Casinos 11
Abbildung 12 - Ablauf der Login-Funktion 11
Abbildung 13 - Animation des Spiels Roulette 12
Abbildung 14 - API-Aufruf mit Axios 13
Abbildung 15 - Erfolgreicher Start des Webservices 14